

Non-deterministic Finite Automata

We now consider an extension of DFA, by introducing nondeterminism in its transition fn. The resulting model is called Nondeterministic Finite Automata (NFA).

In a DFA, transition fn specifies a unique next state from each state and input symbol. In contrast, in NFA transition fn specifies a set of states from any state on any input symbol.

Mathematically, transition fn of DFA has form

$$\delta: Q \times \Sigma \rightarrow Q$$

but transition fn of NFA has form

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

(2^Q is the power set of Q , also written as $P(Q)$ or $\text{Pow}(Q)$)

When a NFA M is in state q and reading a symbol 'a' it may move to any state in $\delta(q, a)$. In other words, next state of M from (q, a) is chosen nondeterministically from the set $\delta(q, a)$.

$x \in L(M)$ iff there is a sequence of (non-deterministic choices of) moves of M on input x which ends in a final state. We make it more precise by defining a computation (or a run) of M on input x .

Defn (run of a NFA)

Let $M = (Q, \Sigma, q_0, \delta, F)$, where $\delta: Q \times \Sigma \rightarrow 2^Q$, be a NFA. Let $x = a_1 a_2 \dots a_n$ be an input, each $a_i \in \Sigma$.

A run of M on x is defined to be a sequence of states $q_0, q_1, \dots, q_n$ s.t. for each $1 \leq i \leq n$, $q_i \in \delta(q_{i-1}, a_i)$.

More pictorially, we may write a run as

$$q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \rightarrow \dots \rightarrow q_{n-1} \xrightarrow{a_n} q_n.$$

A run is said to be accepting if $q_n \in F$.

In a DFA, for each input there is ~~at~~ a unique run.
~~if~~ In contrast in an NFA, for an input there may be several (or none at all) runs.

Defn
 $x \in L(M)$ iff there is an accepting run of M on x .

Remark: we also use the word 'computation' for a 'run'.

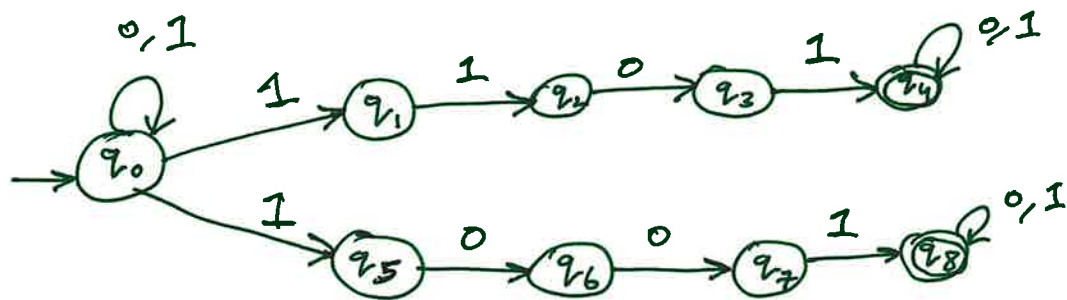
In state diagrams, difference in transition fn from DFA shows up by having ^{possibly} several ~~distinct~~ ^{different} arrows from a state, all ~~are~~ marked by the same input letter. In other words, a transition of NFA is represented by drawing ~~an~~ arrows labeled 'a' ~~from~~ from q to all $p \in \delta(q, a)$.

Acceptance corresponds to a path from ^{the} initial to a final state with ~~s.t.~~ labels of this path when concatenated being the input.

Example 1:

Let $L = \{x \in \{0,1\}^* \mid x \text{ contains } 1101 \text{ or } 1001 \text{ as a substring}\}$

We design following NFA to accept L .



The automaton is non-deterministic, because $\delta(q_0, 1) = \{q_1, q_5\}$.

Also note that $\delta(q_1, 0) = \emptyset$. Correctness is easy to see.

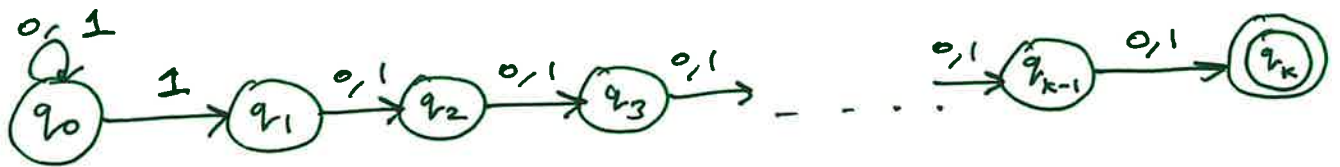
Exercise: The above automaton shows a general scheme for ~~an~~ accepting languages like L . However, for the given L it is not optimal (in no. of states). Show a five states NFA to accept L .

Example 2:

For $k \in \mathbb{N}$, $k \geq 1$ define

$$L_k = \{ x \in \{0,1\}^* \mid \text{the } k^{\text{th}} \text{ last bit of } x \text{ is } 1 \}$$

We design NFA for L_k with $k+1$ states as follows.



We leave the reader to convince himself of ~~the~~^{its} correctness.

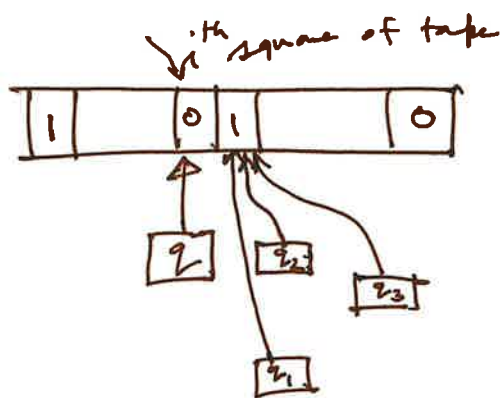
Why nondeterminism in FA?

What is the reason for considering NFA instead of DFA? Previous examples should convince the reader that NFA can be easier and more succinct to design than DFAs.

Exercise:

~~Design~~ Design DFAs for languages in both examples above.

Nondeterminism is primarily a mathematical concept, still one may ask if ~~the~~ there is a way to implement it directly. Closest answer is the following interpretation which involves copying of hardware.



$$\delta(q, 0) = \{q_1, q_2, q_3\}$$

Consider (a copy of) NFA M ~~starting~~ in configuration (q, i) [that is in state q and reading ~~the~~ ^{the} tape square i from the left]

as shown ~~is~~ in above figure.

As $|\delta(q, 0)| = 3$, M makes three copies of itself ^{each copy} which are in states q_1, q_2, q_3 respectively and ~~one~~ moves ~~one~~ square to the right on the tape. These copies work, independently (of each other) from this point on. An input is accepted if any copy ends in a final state after reading the last symbol of the input.

How many copies can be there of M at the end of an n -length input? ~~Suppose~~ suppose, each transition maps to exactly two different states, ~~i.e.~~ i.e. $|\delta(q, a)| = 2$, for each q, a . Then there will be 2^n such copies, however many of these copies may be in the same state as number of distinct states is $|Q|$ which is fixed irrespective of the input size. Copies in the same configuration proceed identically ~~in~~ in future, or rest of the computation, therefore they may be merged into one. So there can be at most $|Q|$ copies of M at any point.

Expressive Power of NFA

Can NFA recognize languages, which no DFA can recognize?
The answer to this question is "no", as shown in the following theorem.

Theorem 1

If L is recognized by a NFA then there is a DFA which also recognizes L .

Proof:

Let $M = (Q, \Sigma, q_0, \delta, F)$ be a NFA

s.t. $L(M) = L$.

The idea of equivalent DFA is to keep track of all the states of M that can be reached ~~that can be reached~~ after reading the input.

It is useful to introduce some notation and terminology.
~~that~~ we say that $q \in Q$ is reachable on input x if there is a run of M starting in q_0 and which ends in state q after reading x . This is denoted succinctly as:  $q_0 \xrightarrow{x}_M q$.

We design a DFA D to accept L as follows.

$$D = (2^Q, \Sigma, \{q_0\}, \delta_D, \{F' \mid F' \cap F \neq \emptyset\})$$

$$\delta_D(\{q_1, \dots, q_k\}, a) = \bigcup_{i=1}^k \delta(q_i, a)$$

We claim that for each $x \in \Sigma^*$

$$\delta_D(\{q_0\}, x) = \{q \mid q_0 \xrightarrow{x}_M q\}$$

The proof is by induction on x .

Base case:

$$x = \varepsilon$$

$$\delta_D(\{q_0\}, \varepsilon) = \{q_0\}$$

(By defn of $\hat{\delta}_D$)

$$= \{q_0 \mid q_0 \xrightarrow{\varepsilon}_M q_0\}$$

(Computation of M on input ε does not change state. only computation on ε is " q_0 ")

Induction step:

$$\text{Let } x = wa$$

$$\text{Note that } q_0 \xrightarrow{wa}_M q$$

$$\Leftrightarrow \exists p \in Q [q_0 \xrightarrow{w}_M p \xrightarrow{a}_M q]$$

$$\Leftrightarrow \exists p \in Q [q_0 \xrightarrow{w}_M p \wedge \delta(p, a) = q]$$

$$\Leftrightarrow \exists p \in Q [\cancel{q_0} p \in \delta_D(\{q_0\}, w) \wedge \delta(p, a) = q]$$

(By I.H.)

$$\Leftrightarrow q \in \delta_D(\delta_D(\{q_0\}, w), a)$$

By defn of δ_D

$$\Leftrightarrow q \in \delta_D(\{q_0\}, wa)$$

By defn of $\hat{\delta}_D$.

Therefore

$$q \in \delta_D(\{q_0\}, wa) \Leftrightarrow q_0 \xrightarrow{wa}_M q$$

$$\Rightarrow \delta_D(\{q_0\}, x) = \{q \mid q_0 \xrightarrow{x}_M q\}$$

Now,

$$x \in L(D)$$

$$\Leftrightarrow \delta_D(\{q_0\}, x) \cap F \neq \emptyset \text{ iff } \{q \mid q_0 \xrightarrow{x}_M q\} \cap F \neq \emptyset$$

iff there is a computation of M on input x which ends in a state in F

$$\text{iff } x \in L(M)$$

(End of proof of thm) $\square$